



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



CC ZG506

Lecture No. 11

Agenda

Part-B: Language Models

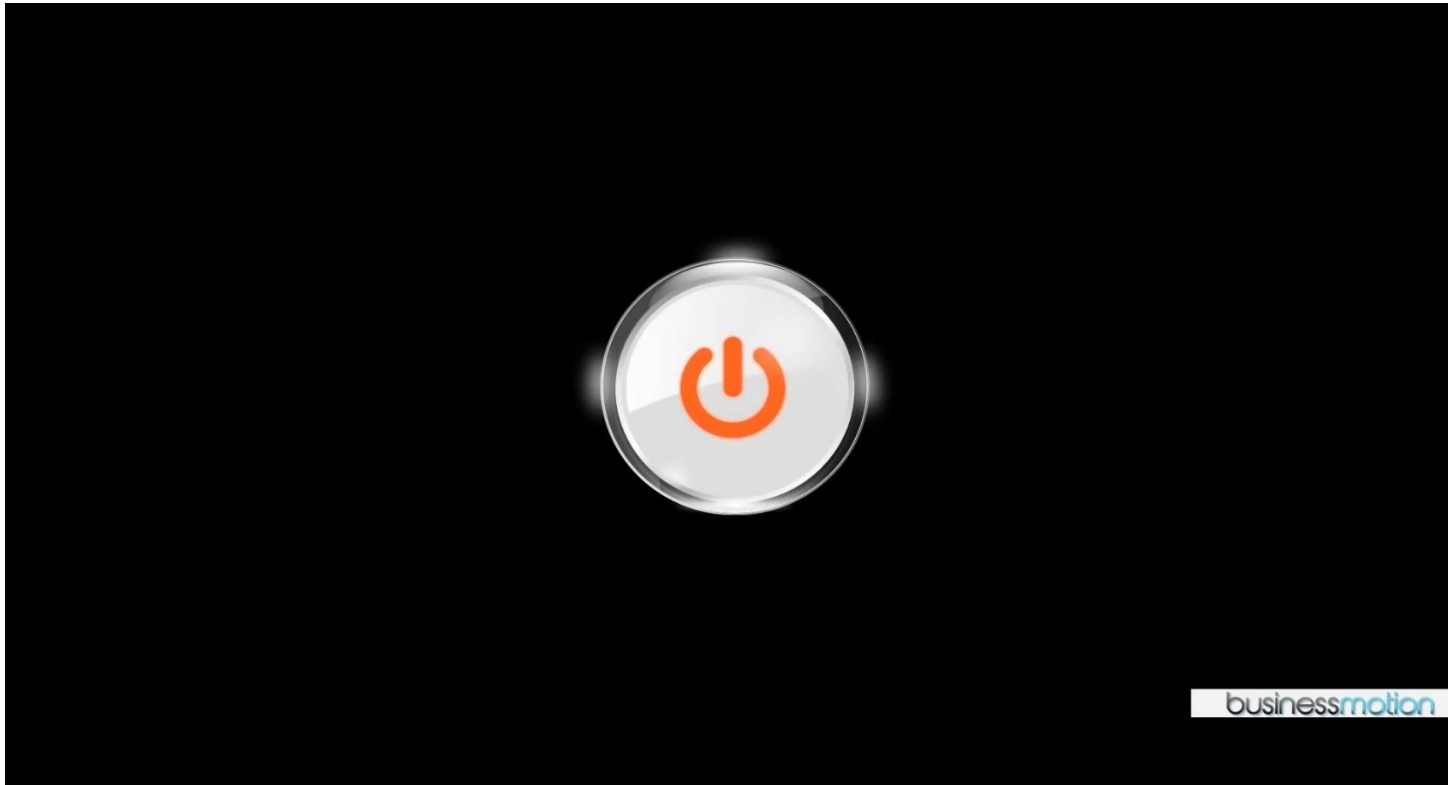
- RAG (Retrieval-Augmented Generation)
 - Document Loading
 - Document Splitting
 - Vectorstores and Embedding
 - Retrieval mechanisms
 - Question Answering
 - Chatbot – design and development



RAG

[Retrieval-Augmented Generation]

What is RAG? By Don Woodlock



<https://www.youtube.com/watch?v=u47GtXwePms>

RAG

- Retrieval-Augmented Generation, or RAG, represents a cutting-edge approach to Artificial Intelligence (AI) and Natural Language Processing (NLP).
- At its core, RAG LLM is an innovative framework that combines the strengths of **retrieval-based** and **generative models**, revolutionizing how AI systems understand and generate human-like text.

<https://www.analyticsvidhya.com/blog/2023/09/retrieval-augmented-generation-rag-in-ai/>

Why RAG?

- The development of RAG is a direct response to the limitations of Large Language Models (LLMs) like GPT.
- While LLMs have shown impressive text generation capabilities, they often struggle to provide contextually relevant responses, hindering their utility in practical applications.
- Other approaches such as “Fine-Tuning” and “Context-based (Prompt engineering using context)” when applied for internal organizational data, have limitations or restrictions when it comes to volume(s) of data (GBs or TBs) fed as input to the prompt.
- RAG LLM aims to bridge this gap by offering a solution that excels in understanding user intent and delivering meaningful and context-aware replies.

<https://www.analyticsvidhya.com/blog/2023/09/retrieval-augmented-generation-rag-in-ai/>



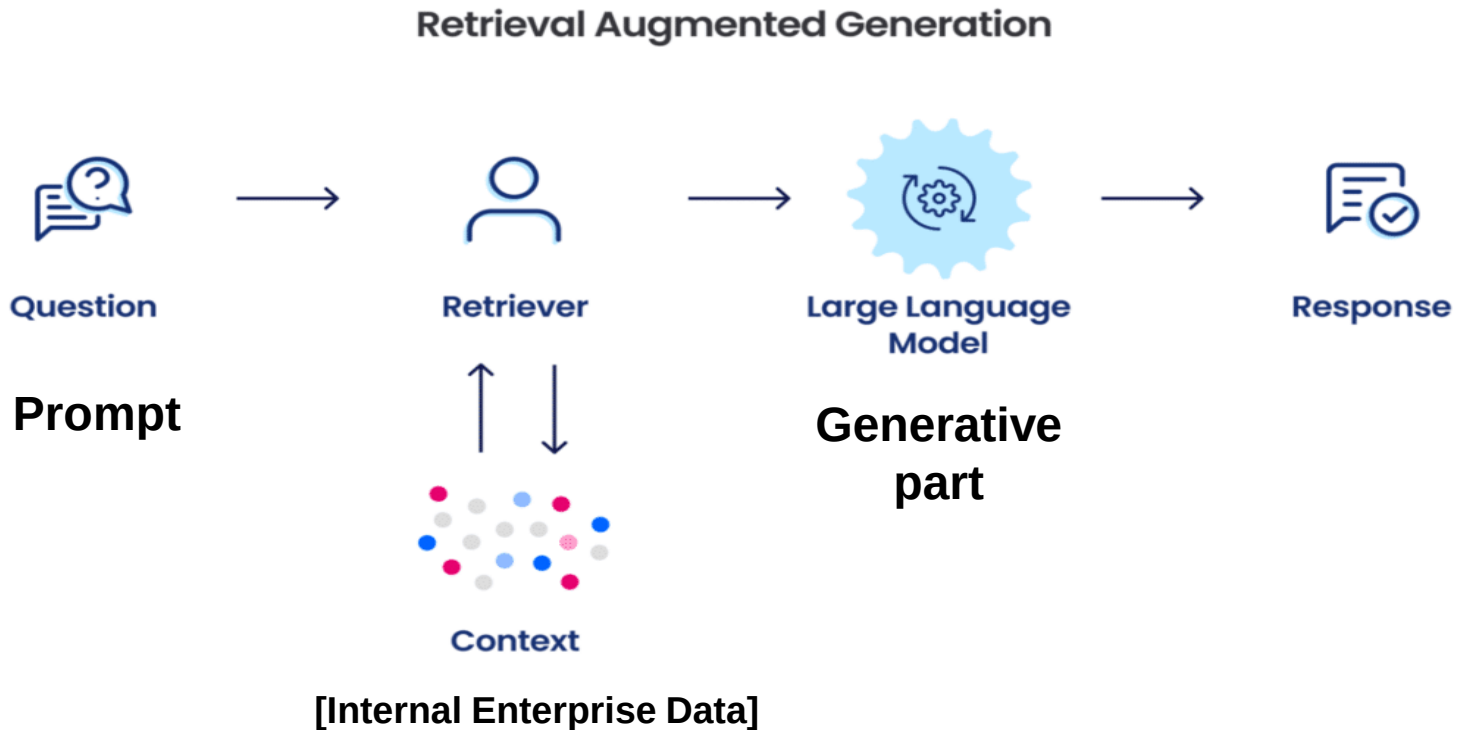
Terminologies

- **Prompt** -> Refers to the user's input or question, which is the initial query that drives the model's response.
- **Context** -> The information or data the system refers to when generating a response. This can include both internal data (pre-loaded knowledge in the model) and external data (additional documents or information pulled in real-time).
- **LLM** -> Refers to Open AI's ChatGpt models (3.5, 4, 4o, 4o-mini) or LLama3 etc.

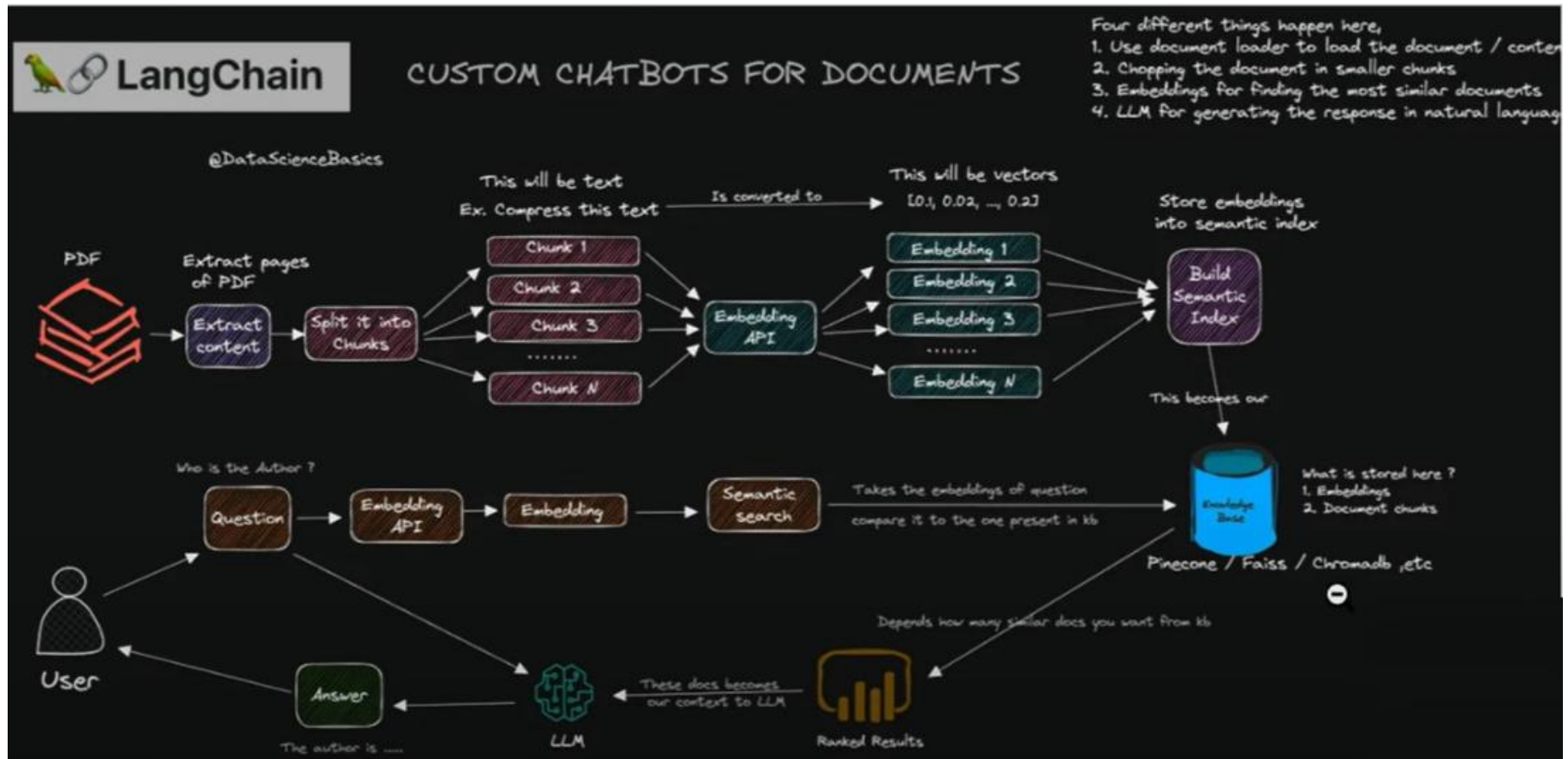
Terminologies

- **Retriever** -> is the component that searches a knowledge base (Context) to find relevant information or documents based on the user's query.
- **Embedding**: An embedding is a numerical representation of text data, typically as a vector of numbers, that captures the semantic meaning, enabling the model to understand similarities between words or phrases.
- **Vector Store**: A vector store is a specialized database that stores embeddings (vectors) and allows efficient similarity search, enabling quick retrieval of relevant information based on proximity in vector space.

RAG Architecture [Simplistic]



RAG Model [Detailed Architecture]



Embedding Types

- **OpenAI Embeddings:** Embeddings generated by OpenAI's models, suitable for a wide range of NLP tasks.
- **BERT Embeddings:** Derived from the BERT model, capturing semantic meaning and context in text.
- **Sentence Transformers (e.g., Sentence-BERT):** Specially optimized for sentence and document embeddings, capturing semantic relationships effectively.
- **FAIR's Dense Passage Retriever (DPR):** Optimized for retrieval tasks, commonly used in open-domain question answering.
- **GloVe (Global Vectors for Word Representation):** Pre-trained embeddings that capture semantic relationships between words, though they lack contextual awareness.



Vector Stores

- **Pinecone**: A managed vector database offering scalable and fast similarity search for high-dimensional embeddings.
- **FAISS** (Facebook AI Similarity Search): An open-source library by Facebook optimized for fast, large-scale similarity search.
- **Weaviate**: An open-source vector database with built-in semantic search capabilities, often used in NLP and RAG applications.
- **Milvus**: A high-performance, open-source vector database that supports similarity search and is commonly used for embedding-based retrieval.
- **ElasticSearch** with KNN Plugin: A search engine with a plugin for k-nearest neighbors (KNN) search, enabling vector-based similarity search within text-based indices.

We will use OpenAI Embeddings and FAISS Vector Store for Demo

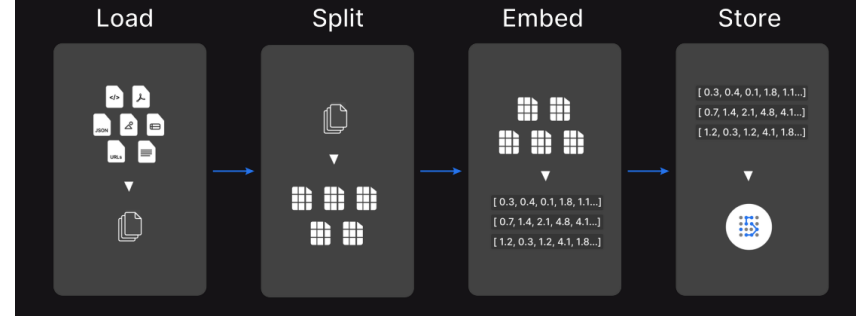
LangChain

- LangChain is an open-source framework designed to streamline the development of applications that integrate with language models, particularly large language models (LLMs) like OpenAI's GPT models.
- It is widely used to build advanced applications such as chatbots, virtual assistants, document summarizers, and any other software that requires natural language processing (NLP) capabilities.

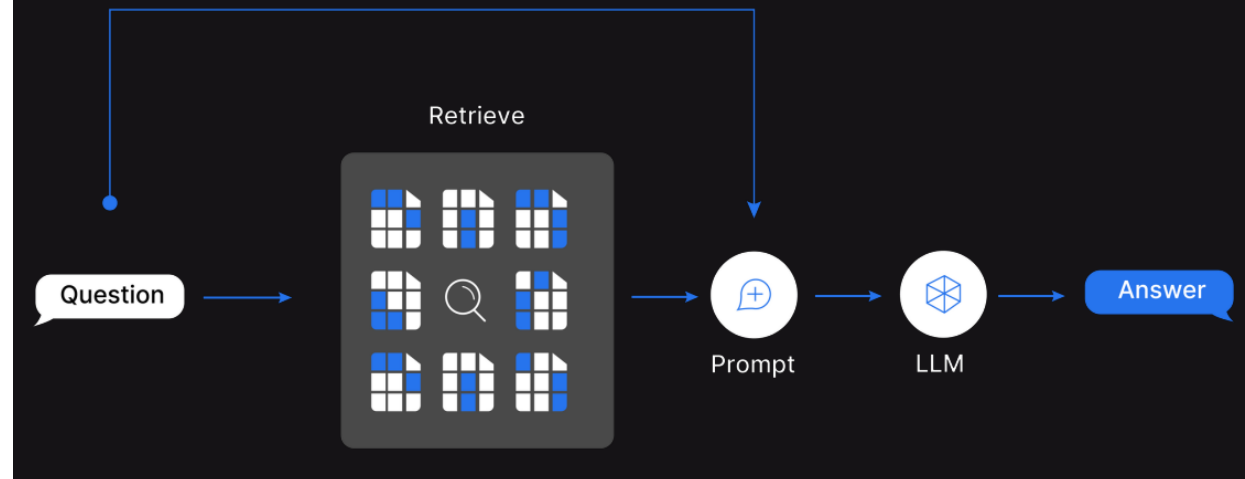
Stages in RAG Model

1. Document Loading
2. Document Splitting
3. Embedding and Vectorstores
4. Retrieval
5. Question Answering
6. Chat

RAG System Architecture - Data Indexing



RAG System Architecture - Search and Generation



RAG Model – Demo

The code takes a PDF document as input using OpenAI's gpt-4 model and Langchain. It breaks the content into chunks, creates embeddings, searches for relevant content, and uses a chain to generate the final summary.

With Gradio Interface: The code operates as a chatbot, where the user can enter multiple queries related to the uploaded document. Each query initiates a new retrieval and summarization process without requiring a new file upload.

Sample Prompts (Tasks)

1. Summarize the document in 100 words
2. What are virtual machines? answer in 40 words

****File:** RAGModel.ipynb

****Input File Upload:** "VirtualMachines.pdf"

1. Document Loading

- The document is loaded when the user uploads a PDF through the Google Colab interface. The code takes the first uploaded file from the dictionary returned by `files.upload()` and writes its content to disk.
- The PDF is read using `PdfReader`, and the text is extracted from each page. The extracted text is combined into a single string called “`raw_text`”, which serves as the full text content for further processing.

2. Document Splitting

- Since large documents can be difficult to process in one go, the code uses `CharacterTextSplitter` to split the `raw_text` into smaller chunks.
- Each chunk has a maximum size of 1000 characters with a 100-character overlap, allowing the model to retain context between segments while staying within token limits.
- Each chunk is then wrapped into a “document” object, which makes it easier to handle and process in later steps.

3. Vectorstores and Embedding

- To perform effective retrieval based on the semantic meaning of the text, each chunk is converted into an "embedding" (a numerical representation of its content) using OpenAIEmbeddings from the LangChain library.
- These embeddings are then stored in FAISS (Facebook AI Similarity Search), a vector storage and similarity search tool, allowing the system to efficiently search through the document chunks to find those most relevant to any query.
- The FAISS index (stored in “docsearch” variable) is initialized once and used for all future queries on this document.

4. Retrieval

- When a query is made, the code retrieves the most relevant chunks of text based on their similarity to the query.
- The `similarity_search` function in FAISS finds the top 10 most relevant document chunks from the vector store, ensuring that only the most relevant parts are passed to the language model.

5. Question Answering

- The retrieved chunks are passed to a summarization chain set up using GPT-4, with the prompt specifying the user's query.
- This `load_summarize_chain` function takes the selected documents and query, instructing the model to provide a summarized response based on the document contents.
- The result is a concise, relevant answer to the query based on the content in the uploaded document.

6. Chatbot

- The code operates as a chatbot, where the user can enter multiple queries related to the uploaded document. Each query initiates a new retrieval and summarization process without requiring a new file upload.
- The chatbot continues to accept queries until the user types "exit," at which point it terminates the session.
- This interactive setup allows users to ask questions about the uploaded document in a conversational format, with the model providing summaries in response to each query.

Key Libraries

- LangChain: Helps with chaining language model tasks.
- FAISS: A library for efficient similarity search.
- PyPDF2: Reads and extracts text from PDF files.
- OpenAI: Provides embeddings and large language models (LLMs).

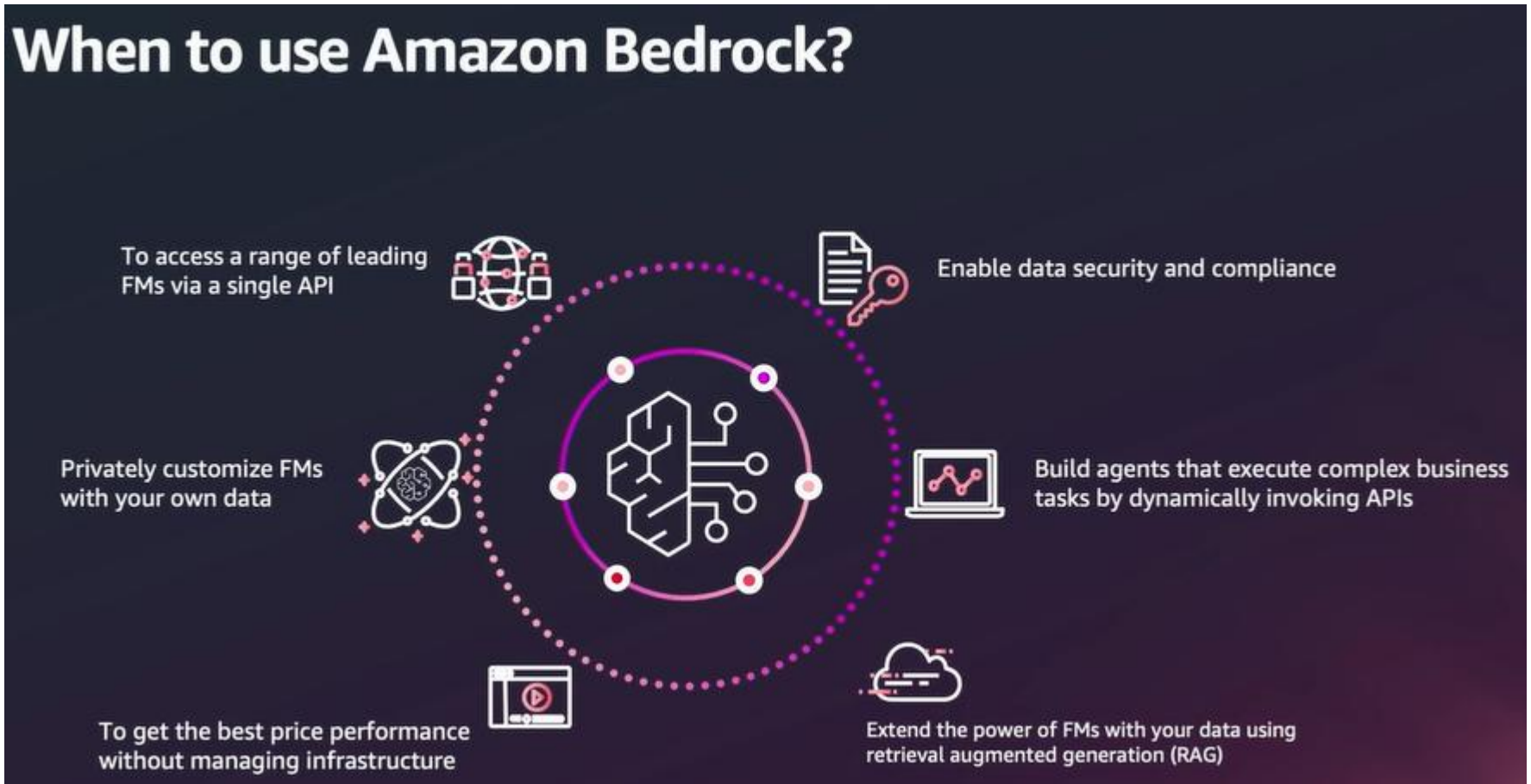


Thank You!

Appendix – Additional Content

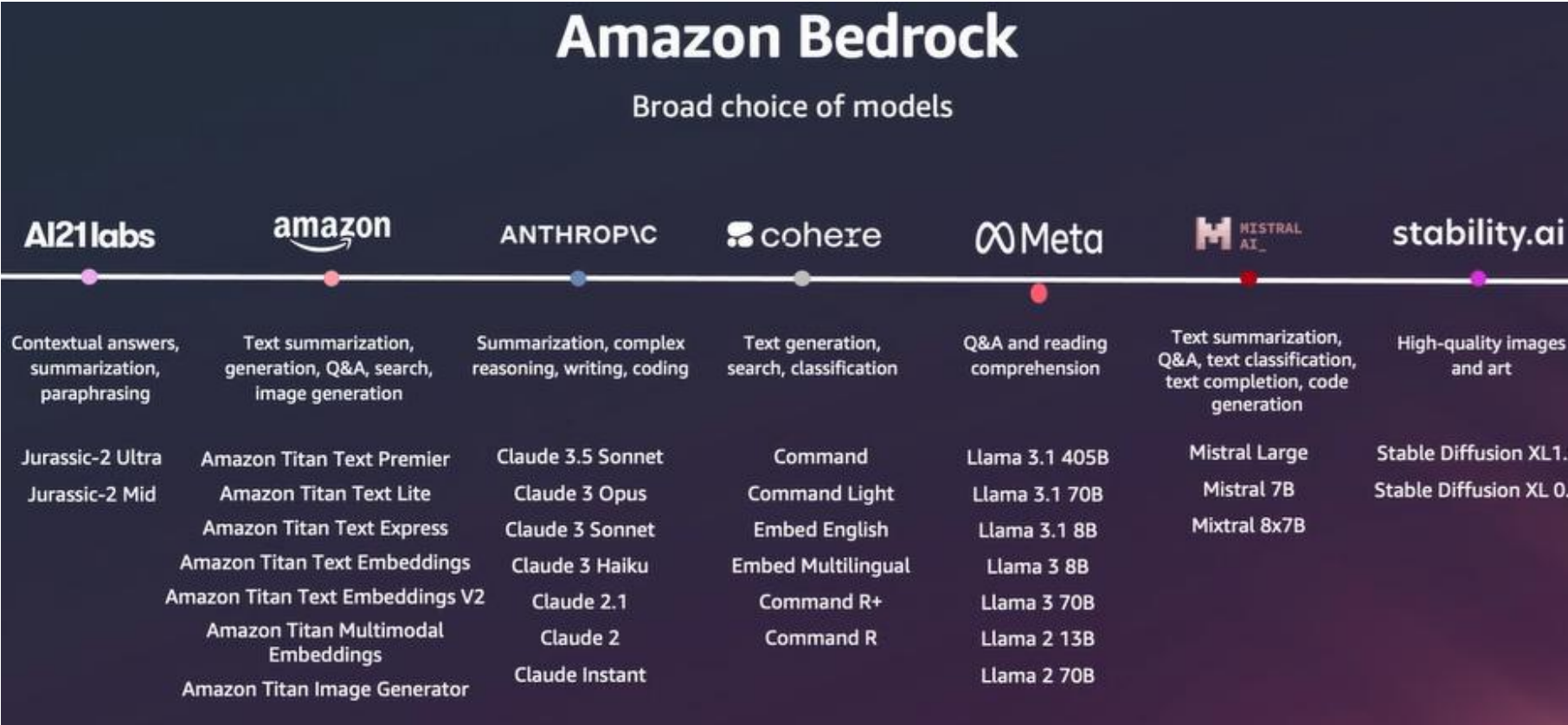
Amazon BedRock

When to use Amazon Bedrock?



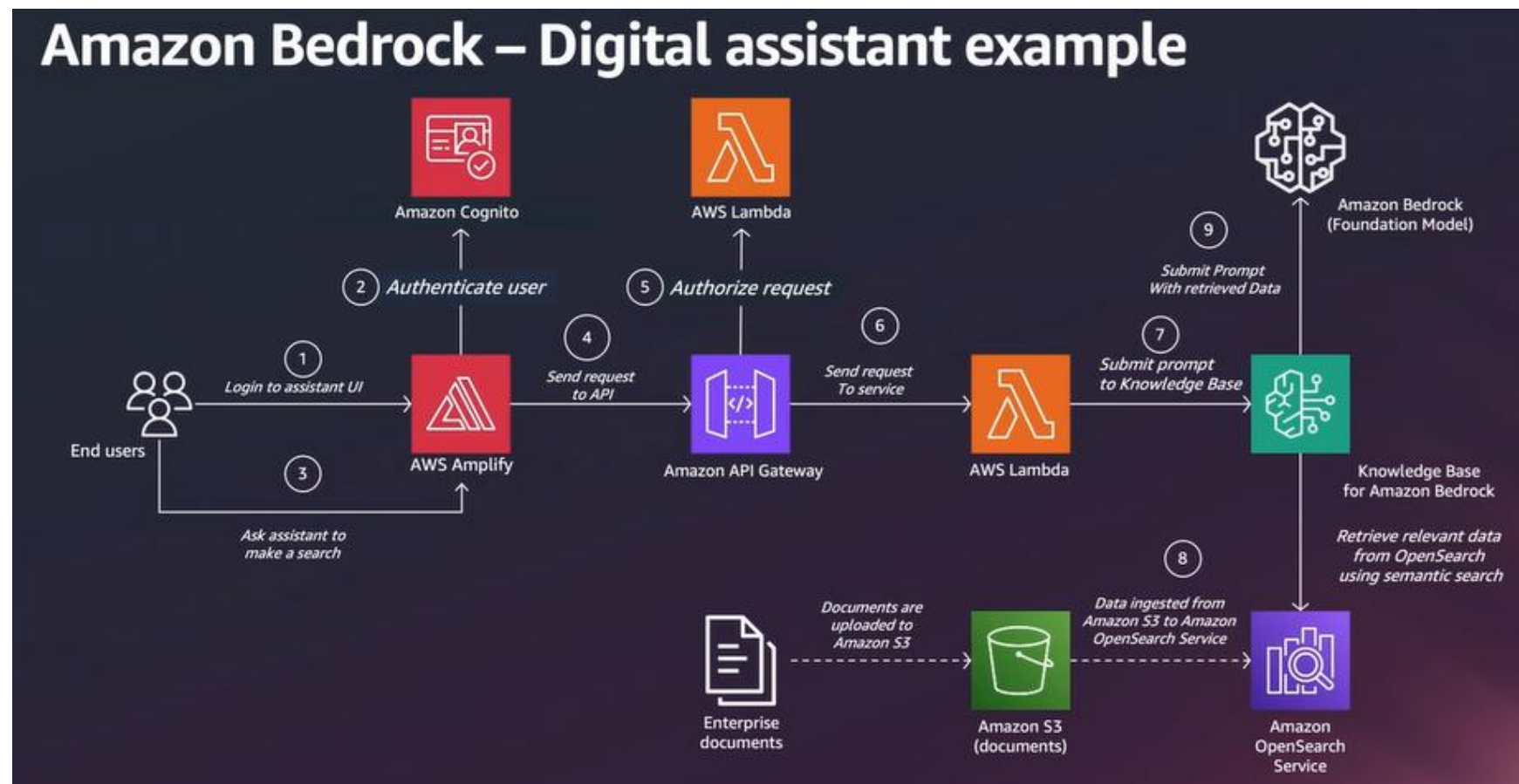
Ref: Adapted from 'AWS Innovate Conference, 2024', Presentation by Melwin Pais, Senior Solution Architect, AWS

Amazon BedRock – Choice of Models



Ref: Adapted from 'AWS Innovate Conference, 2024', Presentation by Melwin Pais, Senior Solution Architect, AWS

Amazon BedRock – Use Case



Ref: Adapted from 'AWS Innovate Conference, 2024', Presentation by Melwin Pais, Senior Solution Architect, AWS